Detección de funciones similares mediante AST

Decisiones de Diseño

Extracción: con os.walk se recorre el directorio, y cada archivo .py se parsea con ast.parse. Con ast.walk obtenemos los nodos de FunctionDef y AsyncFunctionDef, incluyendo los métodos de clases y las funciones anidadas. De cada función se guarda el nombre, archivo, línea y subárbol AST.

Normalización: Se implementa con ast.NodeTransformer sobre una copia del árbol (usando copy.deepcopy), así se conserva el original. Las variables y parámetros se reemplazan por VAR1, VAR2, etc. según el orden que aparezcan, y un mismo identificador recibe siempre el mismo token. De esta forma, podemos mantener la correspondencia de que a+b es igual a x+y, pero no a b+b. Los literales se reemplazan por LIT, el nombre de la función por FUNC, y se eliminan las anotaciones de tipo. Los builtins (len, print…) se conservan porque son parte de lo que hace la función.

Serialización: Mediante un recorrido recursivo en pre-order, se transforma el árbol normalizado en una secuencia lineal de tokens. Por cada nodo visitado se emite su tipo sintáctico (For, BinOp, Return, etc.) y, para los nodos arg, Name y Constant, se adjunta también su token previamente normalizado. Decidimos omitir los nodos de contexto de almacenamiento (Load, Store, Del), ya que son detalles internos del intérprete que no suman información relevante entre las funciones.

Motor de Búsqueda y CLI**:** A través del script principal, se captura la función objetivo bajo el formato archivo.py:funcion y se la compara contra el resto del repositorio. La búsqueda es por fuerza bruta, sin índices: la función objetivo se compara contra todas las del repositorio (O(n) comparaciones por búsqueda), excluyendo a la propia función. Detectar todos los duplicados del repositorio implicaría repetir esto para cada función ($O(n^2)$), lo cual el enunciado considera aceptable para un universo de decenas de archivos. El sistema devuelve el top-k de resultados con mayor score, marcando visualmente como 'candidata a duplicado' a aquellas coincidencias que superen el umbral establecido. La herramienta permite parametrizar por consola la métrica de cálculo (--metrica), el tamaño del n-grama (--ngramas) y el valor del umbral (--umbral).

Métrica elegida y umbral

- Jaccard sobre n-gramas (elegida): proporción de n-gramas compartidos sobre el total. Cada n-grama se guarda junto con su número de aparición, de modo que los repetidos cuentan más de una vez (Jaccard sobre multiconjunto).
- Levenshtein normalizada: 1 − distancia / longitud mayor, con la distancia como mínimo de inserciones, eliminaciones y sustituciones de tokens.

Elegimos Jaccard como métrica principal porque separa con más margen las funciones distintas (entre 0.04 y 0.14 en nuestros tests; score entre 0 y 1). Detectó los clones con variables renombradas (1.00) y el desvío estándar reordenado (0.85), pero no el promedio reordenado (0.57): al ser una función corta, el cambio de numeración de las variables afecta a la mayoría de los trigramas. Levenshtein sí lo detecta (0.84), por eso lo pusimos como alternativa.

Umbral de 0.75. Se fijó como punto de partida (compartir al menos ¾ de los n-gramas) y se validó con los tests: los casos positivos lo superan con la métrica correspondiente y los negativos quedan muy por debajo.

Justificación de n = 3

- n = 1 son tokens sueltos y se pierde el orden. Dos funciones que usen los mismos tipos de nodo, como Name, Assign, For, van a parecer similares, incluso estando armadas distinto.
- n = 2 se pueden formar pares como Name VAR1 o Constant LIT, que pueden aparecer en la mayoría de funciones. Existirían muchas coincidencias entre funciones que no son para nada iguales.
- n = 3 ya captura un patrón chico de estructura, por ejemplo AugAssign Name VAR2 (una acumulación) o Return Name VAR1. Es aproximadamente "una instrucción".
- n grande: Cada token participa en n n-gramas distintos, y un solo cambio rompe muchos n-gramas. Es decir que termina siendo muy estricto.

Limitaciones conocidas

- Diferentes estructuras de control: Si dos funciones resuelven exactamente el mismo problema matemático, pero una utiliza un bucle for y la otra un bucle while, el analizador generará ramas diferentes en el árbol de derivación. Al tener distintos nodos, la similitud bajará drásticamente aunque hagan lo mismo.
- Código de depuración: Si se copia una función pero se le intercala varias instrucciones print() para ver qué valores van tomando las variables, el árbol resultante tendrá muchos más nodos. Esto diluye la proporción de n-gramas compartidos, bajando el puntaje final y pudiendo evadir el umbral de detección.
- Reordenamiento de líneas: como las variables se numeran por orden de aparición (VAR1, VAR2…), intercambiar dos líneas cambia la numeración y rompe los n-gramas. En funciones cortas Jaccard no detecta el clon (0.57 en el test del promedio reordenado).

Posibles mejoras

- Canonicalización de árboles: Previo a la etapa de serialización, se podría agregar un paso extra de transformación para unificar constructos equivalentes hacia una forma canónica común. Por ejemplo, normalizar todas las asignaciones aumentadas (AugAssign, como x += 1) a asignaciones estándar (Assign, como x = x + 1), u ordenar alfabéticamente los operandos de operaciones matemáticas conmutativas (sumas o multiplicaciones) para que los árboles coincidan sin importar el orden en que el programador escribió los factores, o numerar las variables de forma independiente del orden en que aparecen.
- Optimización de la búsqueda: Aunque la búsqueda por fuerza bruta es suficiente para repositorios chicos, si el volumen de archivos creciera mucho el rendimiento decaería, sobre todo para detectar todos los duplicados del repositorio ($O(n^2)$). Esto podría solucionarse implementando indexación mediante MinHash o Locality-Sensitive Hashing (LSH), técnicas que permiten agrupar firmas de n-gramas similares en buckets de memoria, comparando únicamente las funciones que caen en el mismo grupo en lugar de recorrer todo el repositorio.